Programming for Robotics: Project Report

Project Title: Voom-Bot: Autonomous Indoor Cleaning Robot

GitHub link: https://github.com/SpookySaikiK/3003ICT-Group-7-Project

Group Members:

- Lucas El-Harrif – s5342319
- Michelle Murutsi – s5299221
- Tashinga Blessed Nyamadzawo – s5415355

Specialisation Option Chosen: Autonomous Robot (Webots)

1. Project Overview

Voom-Bot is an autonomous indoor cleaning robot developed in Webots using the Pioneer 3-DX platform. The purpose of the robot is to autonomously explore, map, and clean indoor environments such as classrooms, offices, libraries, and living spaces while safely avoiding hazards and obstacles. The robot solves the problem of autonomous floor coverage in unknown indoor environments without requiring manual control. It performs exploration, obstacle avoidance, cleaning coverage, cliff detection, docking, and battery management using a structured finite state machine (FSM). The system integrates multiple sensors and actuators to demonstrate intelligent robotic behaviour.

Sensors Used

- 16 proximity sensors for obstacle detection
- Cliff sensors for edge detection
- GPS for localisation
- Compass for orientation tracking
- Keyboard input for manual user interaction

Actuators Used

- Differential drive wheel motors
- LED indicators
- Speaker output

Intelligent Behaviours Implemented

- Autonomous wall-following exploration
- Grid-based environment coverage mapping
- Perception-driven obstacle avoidance
- Automatic return-to-dock charging behaviour
- Stuck detection and recovery mode
- Cliff avoidance safety behaviour

The robot operates inside a simulated indoor Webots environment with walls, obstacles, and drop off hazards.

2. Technical Requirements Compliance

Core Requirements

Requirement	Implementation
At least 2 inputs	Proximity sensors, cliff sensors, GPS, compass, and keyboard input
At least 2 outputs	Wheel motors, LEDs, and speaker
FSM or Behaviour Tree	Finite State Machine implemented with 7 states
Minimum 4 Behavioural states	PAUSED, EXPLORE, CLEANING, AVOID, CLIFF, RECOVERY, CHARGING
Multi-condition decision logic	State transitions depend on battery level, obstacle detection, cliff detection, exploration completion and user input
Safety or fail-safe mechanism	Cliff avoidance, obstacle avoidance, low battery charging, stuck detection, recovery

Autonomous Robotics Requirements (Option B)

Requirement	Implementation
Navigation	GPS and compass based navigation with grid target movement
Obstacle Avoidance	Reactive turning using proximity sensor data
Perception-driven decisions	Sensor readings directly influence state transitions and robot behaviour

3. System Architecture

The system follows a structured Sense -> Process -> Decide -> Act architecture.

Sense

The robot continuously gathers environmental information using:

- Proximity sensors
- Cliff sensors
- GPS
- Compass
- Keyboard input

Process

Sensor data is processed to:

- Detect nearby obstacles
- Detect cliffs or ledges
- Track robot position and orientation
- Update the occupancy grid map
- Monitor battery level
- Detect stuck conditions

Decide

The finite state machine determines the robot's next behaviour based on:

- Obstacle conditions
- Battery status
- Exploration completion
- Safety overrides
- User commands

Act

The robot performs actions using:

- Wheel motors for movement
- LEDs for state indication
- Speaker alerts during recovery mode

Information Flow

1. Sensors provide raw environmental data.
2. Data is filtered and interpreted.
3. FSM evaluates conditions and selects behaviour.
4. Motor commands and outputs are generated.
5. The loop repeats continuously during simulation.

4. Behaviour Model

The robot behaviour is controlled using a Finite State Machine (FSM). FSMs were chosen because they provide clear behavioural separation, predictable transitions, and simplified debugging.

Behavioural States

State	Purpose
PAUSED	Robot remains Idle
EXPLORE	Wall following exploration
CLEANING	Grid based coverage
AVOID	Reactive obstacle avoidance
CLIFF	Emergency cliff avoidance
RECOVERY	Handles stuck situations
CHARGING	Returns to dock and recharges

State Transition Conditions

EXPLORE -> CLEANING

Triggered when exploration is completed and battery is sufficient.

Any State -> CLIFF

Triggered immediately when cliff sensors detect a drop.

CLEANING -> CHARGING

Triggered when battery becomes low or cleaning is complete.

Any Active State -> RECOVERY

Triggered when the robot remains stationary for too long.

AVOID -> Previous Behaviour

Triggered when no obstacle is detected.

CHARGING -> PAUSED

Triggered when battery reaches maximum charge.

The FSM structure improves modularity and ensures safety critical behaviours override normal operation.

5. Perception / Sensor Processing

The robot combines multiple sensor inputs to make intelligent navigation decisions.

Obstacle Detection

The robot uses 16 proximity sensors. Each sensor contributes weighted influence values to left and right turning forces.
The read_obstacles() function:

- Reads all sensor values
- Converts readings into estimated distances
- Applies directional weighting
- Produces left and right obstacle force values

These values determine turning direction and obstacle avoidance behaviour.

Cliff Detection

Two downward-facing cliff sensors detect ledges or stairs. If either sensor reading drops below the cliff threshold:

- The robot reverses
- Rotates away from danger
- Enters the CLIFF state

Localisation

GPS and compass data provide:

- Robot position (x, z)
- Heading angle theta

This information supports:

- Dock navigation
- Coverage mapping
- Target movement

Grid-Based Mapping

The environment is divided into grid cells.
The system:

- Marks visited cells
- Tracks exploration coverage
- Selects unvisited cleaning targets using zigzag traversal

Coverage percentage is continuously calculated.

Multi-Condition Decision Logic

Robot decisions depend on combinations of:

- Obstacle presence
- Battery level
- Cliff detection

- Exploration completion
- Dock proximity
- Keyboard commands

Safety conditions always override normal cleaning behaviour.

6. Safety and Robustness

Several safety mechanisms were implemented to improve reliability.

Collision Avoidance

Obstacle sensors continuously monitor nearby objects. When thresholds are exceeded:

- The robot enters AVOID mode
- Turning direction depends on sensor weighting

Cliff Protection

Cliff sensors prevent falls from edges or stairs.
If a cliff is detected:

- Motors reverse immediately
- The robot rotates away from danger

Battery Protection

The robot monitors battery level continuously.
When battery falls below the low threshold:

- Cleaning is interrupted
- The robot navigates back to the dock

Stuck Detection

The robot compares movement distance over time.
If movement remains below a minimum threshold for too long:

- RECOVERY mode activates
- Motors stop
- Speaker alerts are played

Fail-Safe Behaviour

Safety states such as CLIFF and RECOVERY override all normal behaviours to prioritise system protection.

7. Code Structure

The project is divided into modular Python files to improve readability and maintainability.

File	Purpose
main_controller.py	Main control loop and FSM logic
config.py	System constants and configuration values
sensors.py	Sensor initialisation and processing
navigation.py	Navigation and movement behaviours
mapping.py	Grid mapping and coverage logic
utils.py	Shared utility functions

Main Control Loop

The controller repeatedly performs:

1. Sensor updates
2. State transition checks
3. Behaviour execution
4. Motor control output
5. Battery updates

This structure follows standard robotic control architecture.

Key Libraries Used

- math
- controller

8. Key Code Snippets

FSM State Transition Logic

```
if battery <= BATTERY_LOW and state not in (CHARGING, RECOVERY, CLIFF, AVOID):
    charging_complete = False
    state = CHARGING
```

This logic ensures the robot safely returns to the dock when battery becomes low.

Obstacle Detection Processing

```
for i, sensor in enumerate(sensors):
    value = sensor.getValue()
    if value <= 0:
        continue

    distance = 5.0 * (1.0 - value / 1024.0)
    if distance >= 0.5:
        continue

    influence = 1.0 - (distance / 0.5)

    wl, wr = WEIGHTS[i]
    left_force += wl * influence
    right_force += wr * influence
```

This weighted perception system converts sensor readings into directional turning behaviour.

Cliff Detection Safety Override

```
if cliff_detected and state != CLIFF and state != RECOVERY:
    state = CLIFF
```

Cliff detection immediately overrides all other behaviours to protect the robot.

Target Navigation

```
target_angle = math.atan2(dz, dx)
angle_error = wrap_angle(target_angle - theta)

if abs(angle_error) > 0.2:
    turn = clamp(KP_ANGLE * angle_error, -MAX_SPEED, MAX_SPEED)
    return -turn, turn, False
```

This code aligns the robot toward cleaning targets using proportional heading control.

9. Results / Demonstration Summary

The robot successfully demonstrated:

- Autonomous exploration of indoor environments
- Reactive obstacle avoidance
- Cliff detection and prevention
- Grid based cleaning coverage

- Automatic return to dock charging
- Battery aware behaviour transitions
- Stuck detection and recovery alerts

The FSM produced stable behaviour transitions and reliable autonomous operation.

Limitations

- The robot uses simple wall following instead of full SLAM navigation.
- Mapping accuracy depends on GPS precision and cell size configuration.

Challenges Encountered

- Balancing obstacle avoidance responsiveness
- Preventing oscillation during wall following
- Managing smooth transitions between exploration and cleaning state

10. Individual Contributions

Member	Contribution
Lucas El-Harrif	FSM implementation, battery system, docking logic, documentation, obstacle avoidance, mapping system
Michelle Murutsi	FSM implementation, testing, debugging, sensor processing, documentation
Tashinga Blessed Nyamadzawo	FSM implementation, testing, debugging, video demonstration, documentation, mapping system